



CI/CD

Guía Integral de Arquitectura:
Scripts, Docker, GitFlows y
GitHub Actions

**Departamento de TIC · Ingeniería de
Software**

Universidad Icesi · 2026

01

Módulo 1: Scripts de Automatización

02

Módulo 2: Contenedores con Docker

03

Módulo 3: Flujos de Git (Git-Flow)

04

Módulo 4: CI/CD con GitHub Actions

Sección 01

Scripts de Automatización

Principios de ejecución defensiva, validación de parámetros y robustez

- **set -e:** Detiene el script ante el primer comando con código de salida no cero.
- **set -u:** Lanza error si se intenta expandir una variable no inicializada.
- **set -o pipefail:** Propaga códigos de error en tuberías (`cmd1 | cmd2`).
- **trap on_exit:** Limpieza automática de archivos temporales al terminar.

Principio de Robustez

En servidores y pipelines automatizados, un script jamás debe continuar ejecutándose ante fallos silenciosos.

Parámetros y Variables

- Validación de argumentos posicionales obligatorios (\$1, \$2).
- Asignación de valores por defecto: `${APP_ENV:-dev}`.
- Carga segura de variables de entorno desde archivos `.env`.

Verificaciones Previas

- Verificación de herramientas instaladas con `command -v`.
- Comprobación de disponibilidad de puertos y bases de datos.
- Validación de permisos y espacio en disco disponible.

Códigos de Salida

- **exit 0:** Ejecución exitosa y condiciones cumplidas.
- **exit 1..255:** Códigos de error específicos para depuración.
- Uso de constantes con nombres descriptivos de errores.

Formato de Mensajes

- Diferenciación entre mensajes informativos y de error (`stderr`).
- Salidas con marcas de tiempo y niveles (`INFO`, `WARN`, `ERROR`).
- Formato legible y compatible con sistemas de agregación de logs.

set -euo pipefail

Estándar de la Industria

“La automatización defensiva previene el error humano y garantiza ejecuciones determinísticas en cualquier infraestructura.”

02

Contenerización con Docker

Multi-Stage Builds para Spring Boot y React, Compose y Reverse Proxy

- **Etapas 1 (Build):** Imagen JDK completa con Gradle o Maven para compilar el archivo JAR.
- **Etapas 2 (Runtime):** Imagen JRE mínima de ejecución (Eclipse Temurin Alpine).
- **Seguridad:** Creación y uso de un usuario no privilegiado (USER appuser).
- **Resultado:** Imagen final ligera (¡ 150MB) sin compiladores ni código fuente.

Aislamiento de Construcción

El código fuente y las herramientas de compilación quedan aislados de la imagen final desplegada en producción.

- **Etapas 1 (Build):** Contenedor Node.js para instalar dependencias (`npm ci`) y empaquetar con Vite.
- **Etapas 2 (Runtime):** Servidor web Nginx Alpine ultraligero (¡ 25MB).
- **Configuración SPA:** Redirección de rutas HTML5 (`try_files $uri /index.html`).
- **Rendimiento:** Servido de estáticos optimizado con compresión gzip habilitada.

Eficiencia de Despliegue

La imagen final no incluye Node.js ni la carpeta `node_modules`, reduciendo drásticamente la superficie de ataque.

Definición de la Arquitectura

- **Servicio DB:** PostgreSQL con volúmenes persistentes nombrados.
- **Servicio API:** Spring Boot conectado a la BD mediante variables `.env`.
- **Servicio Web:** Frontend React sirviendo la interfaz de usuario.

Aislamiento de Redes

- Creación de una red bridge interna para comunicación inter-servicios.
- Resolución DNS interna por nombre de servicio (e.g. `postgres-db:5432`).
- Mapeo exclusivo de puertos públicos necesarios hacia el exterior.

Punto de Entrada Unificado

- Recepción centralizada de tráfico HTTP/HTTPS en el host.
- Terminación SSL/TLS segura y gestión de certificados.
- Enrutamiento por prefijo de ruta o subdominio.

Mapeo de Rutas de la Aplicación

- `/`: Reenvío al contenedor del Frontend React (puerto 80).
- `/api/`: Reenvío al contenedor de la API Spring Boot (puerto 8080).
- Inyección de cabeceras `Host`, `X-Real-IP` y `X-Forwarded-For`.

Microservicios

Portabilidad y Aislamiento

“Contenerizar cada capa de la aplicación desacopla el desarrollo de la infraestructura y simplifica la escalabilidad.”

Sección 03

Estrategias de Git y GitFlow

Topología de ramas, versionado semántico y control de calidad

Ramas Permanentes

- **main / master:** Representa el estado en producción. Contiene solo releases estables y etiquetados.
- **develop:** Rama de integración continua donde convergen las nuevas funcionalidades terminadas.

Ramas de Soporte Temporal

- **feature/*:** Desarrollos individuales nacidos de develop.
- **release/*:** Preparación y estabilización de versiones.
- **hotfix/*:** Corrección urgente de bugs críticos en producción.

- **MAJOR (X.0.0):** Cambios incompatibles que rompen contratos de API o modelos de datos.
- **MINOR (1.X.0):** Incorporación de nuevas funcionalidades retrocompatibles.
- **PATCH (1.0.X):** Corrección de errores y bugs sin alterar el comportamiento general.
- **Sufijos:** -SNAPSHOT para ramas de desarrollo y -RC1 para candidatos a release.

Contrato de Versiones

SemVer permite a los clientes y consumidores de un API predecir el impacto de actualizar una dependencia o servicio.

Tipos de Commit Estandarizados

- **feat:** Nueva funcionalidad en backend o frontend.
- **fix:** Corrección de un fallo o error reportado.
- **refactor:** Mejora estructural del código sin cambiar comportamiento.
- **ci / docs / test:** Tareas de soporte y pipeline.

Estructura del Mensaje

- Encabezado conciso en modo imperativo con alcance opcional.
- Cuerpo descriptivo explicando el *por qué* del cambio.
- Referencias a tickets, issues o cambios disruptivos (BREAKING CHANGE).

- **Protección de Ramas:** Restringir el push directo a `main` y `develop`.
- **Pruebas Automatizadas:** Ejecución obligatoria del pipeline de CI antes de autorizar el merge.
- **Revisión por Pares:** Aprobación requerida de al menos un revisor del equipo de desarrollo.
- **Estrategia de Merge:** Uso de *Squash and Merge* para mantener un historial limpio y lineal.

Cultura de Calidad

El Pull Request es el punto focal de colaboración técnica, detección temprana de deuda técnica y transferencia de conocimiento.

Branch Protection

Disciplina en Repositorios

“Ningún cambio ingresa a las ramas principales sin validación automatizada de pruebas y revisión de código.”

04

CI/CD con GitHub Actions

Pipelines declarativos, workflows condicionales y entornos de ejecución

- **Ubicación:** Archivos YAML estructurados bajo `.github/workflows/`.
- **Triggers (on):** Eventos de activación por `push`, `pull_request` o manuales (`workflow_dispatch`).
- **Jobs:** Conjunto de tareas que se ejecutan en paralelo o de forma secuencial con dependencias (`needs`).
- **Steps:** Pasos individuales que invocan acciones de la comunidad (`uses`) o comandos de shell (`run`).

Pipeline as Code

Toda la lógica de integración, pruebas y despliegue vive versionada en el mismo repositorio del proyecto.

Fase de Backend (Spring Boot)

- Configuración del JDK mediante `actions/setup-java`.
- Ejecución de pruebas unitarias y de integración (`mvn test` / `gradle test`).
- Análisis estático de código y reporte de cobertura de pruebas.

Fase de Frontend (React)

- Instalación determinística con `npm ci`.
- Validación de sintaxis y estilos con Linters (ESLint).
- Ejecución de pruebas unitarias de componentes con Jest o Vitest.

Validación en Pull Requests

- Se activa ante `pull_request` hacia `develop` o `main`.
- Ejecuta compilación y pruebas sin realizar despliegues en servidores.
- Bloquea la integración si alguna prueba unitaria falla.

Despliegue Continuo (CD)

- Se activa únicamente tras el merge exitoso a `main` o `release/*`.
- Construye las imágenes Docker optimizadas de la API y el Frontend.
- Despliega los nuevos contenedores y valida la salud del servicio.

- **GitHub-Hosted Runners:** Máquinas virtuales efímeras en la nube pública; ideales para pruebas unitarias aisladas.
- **Self-Hosted Runners:** Agentes ejecutándose en servidores propios o institucionales de la universidad.
- **Seguridad de Red:** Conexión por sondeo seguro (*long-polling*) sin exponer puertos públicos de entrada.
- **Acceso Interno:** Despliegue directo sobre redes privadas, sockets de Docker locales y bases de datos internas.

Flexibilidad Híbrida

Se pueden combinar jobs en la nube para análisis de código y jobs en runners locales para despliegue en servidores del laboratorio.

CI / CD

Ciclo de Vida Automatizado

“Un pipeline robusto transforma el código fuente en un servicio en producción de forma rápida, repetible y confiable.”

1. Scripts con set -euo pipefail y validación
2. Dockerfiles Multi-Stage para Spring y React
3. Disciplina en Ramas de GitFlow y SemVer
- 4. Pipelines Declarativos con GitHub Actions**